

Cahier des charges

Plugin IDE de simulation et d'analyse d'exécution de fonctions

1. Contexte du projet

Le projet consiste à développer un **plugin IDE** permettant aux développeurs de **simuler l'exécution d'une fonction directement dans leur éditeur**, sans devoir lancer l'application complète ni écrire un test.

L'outil doit permettre de :

- sélectionner une fonction
- fournir des valeurs pour ses paramètres
- exécuter la fonction dans un environnement sandboxé
- visualiser le **chemin d'exécution**
- visualiser les **valeurs retournées, erreurs, branches prises**
- gérer les **dépendances externes via des mocks ou fixtures**

Le plugin devra être conçu de manière **extensible**, afin de pouvoir supporter plusieurs IDE et plusieurs langages à terme.

2. Objectifs du projet

Objectif principal

Permettre à un développeur de :

1. cliquer sur une fonction dans son IDE
2. ouvrir un panneau de simulation
3. saisir les paramètres
4. exécuter la fonction
5. voir exactement ce qui se passe (branches, retours, exceptions)

Objectifs secondaires

- faciliter la compréhension d'une codebase
- accélérer le debugging
- générer éventuellement des **tests automatisés**
- permettre le **mockage des dépendances externes**

3. Périmètre du MVP

Langages supportés

Première version :

- JavaScript
- TypeScript

Langages futurs possibles :

- PHP
- Python
- Java
- Go

IDE supporté

Première version :

- VS Code

Extensions futures possibles :

- JetBrains (IntelliJ / PhpStorm / WebStorm)

- Neovim
 - autres IDE compatibles
-

4. Fonctionnalités principales

4.1 Sélection de fonction

L'utilisateur peut sélectionner une fonction dans l'éditeur.

Le plugin doit :

- identifier la fonction sous le curseur
- récupérer sa signature
- récupérer son code source

Exemple :

```
function calculatePrice(cart, coupon)
```

4.2 Génération du formulaire de paramètres

Le plugin doit générer automatiquement un formulaire permettant de saisir :

- paramètres primitifs
- objets JSON
- valeurs par défaut

Exemple :

```
cart: { items: [...] }
```

```
coupon: "WELCOME10"
```

4.3 Simulation d'exécution

Le plugin doit :

- exécuter la fonction dans un **sandbox runtime**
- tracer les événements d'exécution

Événements capturés :

- branch (if / switch)
 - call
 - return
 - throw
 - assignation
 - appels externes
-

4.4 Visualisation du chemin d'exécution

L'utilisateur doit pouvoir voir :

- quelles branches ont été prises
- quelles fonctions ont été appelées
- les valeurs retournées
- les erreurs

Exemple :

Path:

```
if (cart.empty) -> false
```

```
if (coupon == "WELCOME") -> true
```

```
return 90
```

4.5 Détection des dépendances externes

Le système doit détecter les appels externes :

- HTTP
- DB
- filesystem
- random
- date/time
- variables d'environnement

Exemple :

externalCall: fetch("https://api...")

4.6 Système de mocks

Si une dépendance externe est rencontrée :

le plugin doit :

1. suspendre l'exécution
2. demander à l'utilisateur de créer un mock

L'utilisateur peut :

- fournir un retour JSON
- charger une fixture

4.7 Fixtures

Les fixtures permettent de simuler des réponses externes.

Exemple :

fixtures/http/user.json

fixtures/db/orders.json

Ces fixtures sont stockées dans le projet :

.project-simulator/

mocks.json

fixtures/

5. Architecture technique

Le projet suit une architecture **hexagonale (ports & adapters)**.

6. Architecture globale

core engine

|

|

language adapters

|

|

runtime adapters

|

|

IDE adapters

7. Structure du projet

Monorepo recommandé :

packages/

core/

moteur principal

protocol/

types et contrats

adapter-tsj/

analyse et instrumentation TS/JS

adapter-node-runtime/

exécution sandbox

ide-vscode/

plugin VS Code

ui/

interface Webview

8. Core engine

Le core est **indépendant de tout IDE**.

Il contient :

- modèle de données
- use cases
- orchestration simulation

Interfaces principales

Navigation dans le code

CodeNavigator

Responsabilités :

- trouver une fonction
- récupérer signature
- récupérer source

Instrumentation

Instrumenter

Responsabilités :

- modifier le code pour injecter des hooks de tracing

Exécution

SandboxRunner

Responsabilités :

- exécuter le code instrumenté
- renvoyer les événements d'exécution

Gestion des mocks

MockStore

Responsabilités :

- charger les mocks
 - sauvegarder les mocks
-

9. Protocol d'événements

Tous les événements suivent un protocole commun.

Exemple :

TraceEvent

Types possibles :

branch

call

return

throw

assign

externalCall

10. Runtime

L'exécution est réalisée dans un sandbox Node.

Technologie recommandée :

- worker_threads

Avantages :

- isolation
 - timeout possible
 - arrêt forcé
-

11. Interface utilisateur

L'UI est réalisée via **Webview VS Code**.

Technologie possible :

- HTML
 - CSS
 - JavaScript
 - éventuellement React
-

Fonctionnalités UI

Le panneau doit afficher :

- formulaire paramètres
 - bouton Run
 - timeline d'exécution
 - affichage des branches
 - affichage des erreurs
-

12. Support multi IDE

Le projet doit être conçu pour supporter plusieurs IDE.

Chaque IDE implémente un adapter :

EditorBridge

Responsabilités :

- récupérer le fichier actif
 - récupérer la position du curseur
 - ouvrir un panneau
 - afficher des messages
-

13. Évolutions futures

Fonctionnalités possibles :

- génération automatique de tests
 - exploration de chemins multiples
 - support multi-langage
 - analyse statique avancée
 - simulation distribuée
-

14. Roadmap de développement

Phase 1 — MVP

- extension VS Code
 - sélection fonction
 - formulaire paramètres
 - exécution sandbox
 - affichage return / throw
-

Phase 2

- instrumentation branches
 - trace events
-

Phase 3

- gestion mocks simples
 - interception Date / random / env
-

Phase 4

- HTTP mocks
 - fixtures persistantes
-

Phase 5

- support DB mocks
 - export test
-

15. Critères de réussite

Le projet est considéré fonctionnel si :

- une fonction peut être simulée depuis l'IDE
 - les paramètres peuvent être fournis
 - l'exécution produit une trace lisible
 - les dépendances externes peuvent être mockées
-

16. Contraintes

- performance acceptable (<2s pour exécution simple)
- isolation sandbox

- architecture extensible
 - compatibilité multi IDE à terme
-

17. Résultat attendu

Un plugin IDE permettant aux développeurs de :

- comprendre rapidement le comportement d'une fonction
 - simuler différents scénarios
 - explorer les chemins d'exécution
 - mocker les dépendances externes
-